

FastAPI & Backend Development

Practice Coding Questions — Sections 1 & 2

Try to solve each task yourself first (write real code, run it with `uvicorn main:app --reload` and test on `/docs`). Hints are given if you're stuck. Sample solutions are in the Answer Key at the end — don't peek early!

SECTION 1 — API, REST & HTTP FUNDAMENTALS

Q1. Basic GET Endpoint

Create a FastAPI app with one endpoint `GET /` that returns `{"message": "Hello World"}`.

■ **Hint:** Use `@app.get("/")` as a decorator above a plain function.

Q2. Path Parameter

Create `GET /square/{number}` that takes an integer path parameter and returns its square, e.g. `/square/4` → `{"number": 4, "square": 16}`.

■ **Hint:** Type-hint the parameter as `int` in the function signature — FastAPI validates it automatically.

Q3. Query Parameters with Defaults

Create `GET /greet` that accepts an optional query parameter `name` (default "Guest") and returns `{"greeting": f"Hello, {name}!"}`. Test both `/greet` and `/greet?name=Ali`.

■ **Hint:** A function parameter not found in the path is automatically treated as a query parameter.

Q4. POST with Request Body (Pydantic Model)

Create a Pydantic model `Product` with fields `name` (str), `price` (float), `in_stock` (bool, default True). Create `POST /products` that accepts this model and returns the received data plus a generated id (use `len` of a fake in-memory list).

■ **Hint:** `class Product(BaseModel): ...` then `def create_product(product: Product): ...`

Q5. PUT vs PATCH

Given an in-memory dict of products keyed by id, implement `PUT /products/{id}` (replace the whole product) and `PATCH /products/{id}` (update only the fields provided). Return 404 if the id doesn't exist.

■ **Hint:** For `PATCH`, make all fields Optional in a separate 'ProductUpdate' model, then only update fields that aren't None.

Q6. DELETE Endpoint + Status Codes

Implement `DELETE /products/{id}` that removes a product and returns a 204 No Content on success, or 404 if not found.

■ **Hint:** Use `raise HTTPException(status_code=404, detail="Product not found")` and set `status_code=204` in the decorator.

Q7. Reading Request Headers

Create `GET /whoami` that reads a custom header `X-User-Role` from the request and returns it, e.g. `{"role": "admin"}`. Return `{"role": "anonymous"}` if the header is missing.

■ **Hint:** Add a parameter like `x_user_role: str | None = Header(default=None)` to your function.

Q8. Design a Response Model

Create a `UserOut` Pydantic model that excludes a 'password' field. Create `POST /register` that accepts a `UserIn` model (with username, password) but returns only `UserOut` data — password must never appear in the response.

■ **Hint:** Set `response_model=UserOut` in the route decorator; FastAPI filters the output automatically even if you return more fields.

SECTION 2 — SETUP, ROUTING & DOCUMENTATION

Q1. Project Setup From Scratch

Starting from an empty folder: create a virtual environment, activate it, install fastapi and uvicorn, and create a minimal main.py with a root endpoint. Write out every terminal command you'd run, in order.

■ **Hint:** `python -m venv venv → activate → pip install fastapi "uvicorn[standard]" → write main.py → uvicorn main:app --reload`

Q2. Nested Path Design

Design (just the routes, no logic needed) a nested-resource API for a blog: users have posts, and posts have comments. Write the route paths + methods for: list a user's posts, get one comment on one post of one user, and create a new comment.

■ **Hint:** *Nesting mirrors ownership: /users/{user_id}/posts/{post_id}/comments/{comment_id}*

Q3. Multiple Routers (APIRouter)

Split a growing app into two files: users.py (with its own APIRouter for /users routes) and main.py (which creates the FastAPI() app and includes the users router). Write both files' key lines.

■ **Hint:** `router = APIRouter(prefix="/users", tags=["users"])` in users.py, then `app.include_router(users.router)` in main.py.

Q4. Dependency Injection for Auth Check

Write a dependency function `verify_token` that reads an 'Authorization' header and raises a 401 HTTPException if it doesn't equal 'Bearer secrettoken'. Apply it to a protected endpoint `GET /admin/data` using `Depends()`.

■ **Hint:** `def verify_token(authorization: str = Header(...)): if authorization != "Bearer secrettoken": raise HTTPException(401, ...)`

Q5. Async Endpoint Simulating I/O

Write an async `def` endpoint `GET /slow-data` that simulates a 2-second database call using `asyncio.sleep`, then returns dummy data. Explain in one sentence why this doesn't block other requests.

■ **Hint:** `import asyncio; await asyncio.sleep(2)` — because it's a non-blocking await, the event loop can serve other requests during the wait.

Q6. Custom Tags & Descriptions for /docs

Add `tags=["products"]`, a summary, and a description to one of your existing endpoints so that it appears clearly organized when you open /docs. Also set a title and version on the FastAPI() app itself.

■ **Hint:** `app = FastAPI(title="My Shop API", version="1.0.0");` then `@app.get(..., tags=[...], summary=..., description=...)`

Q7. Environment-Based Server Config

Modify your app to read a PORT value from an environment variable (default 8000) and print it on startup using an `@app.on_event("startup")` handler (or lifespan). Explain the difference between running with and without `--reload`.

■ **Hint:** `import os; port = os.getenv("PORT", 8000).` `--reload` restarts the process on file changes (dev only); without it, the server runs a stable single process (or multiple workers) for production.

Q8. Testing an Endpoint with Postman (conceptual)

You built `POST /login` which needs a JSON body `{"username":..., "password":...}` and returns a token. Describe, step by step, how you'd test this in Postman (method, URL, body type, and how you'd reuse the returned token in a later authenticated request).

■ **Hint:** Set method `POST`, URL `http://127.0.0.1:8000/login`, Body `→ raw → JSON`; save token from response, then add it as an `Authorization` header (`Bearer`) in the next request — Postman environments/variables can automate this.

ANSWER KEY — SAMPLE SOLUTIONS

Full working code for the core hands-on questions (S1 Q1–Q6, S2 Q1 & Q4). For the rest, use the hints above — writing it yourself is where the learning happens.

Section 1 — Q1 to Q6 (single app)

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()

# Q1
@app.get("/")
def read_root():
    return {"message": "Hello World"}

# Q2
@app.get("/square/{number}")
def get_square(number: int):
    return {"number": number, "square": number ** 2}

# Q3
@app.get("/greet")
def greet(name: str = "Guest"):
    return {"greeting": f"Hello, {name}!"}

# Q4
class Product(BaseModel):
    name: str
    price: float
    in_stock: bool = True

db: dict[int, Product] = {}
next_id = 1

@app.post("/products")
def create_product(product: Product):
    global next_id
    db[next_id] = product
    result = {"id": next_id, **product.dict()}
    next_id += 1
    return result

# Q5
class ProductUpdate(BaseModel):
    name: str | None = None
    price: float | None = None
    in_stock: bool | None = None

@app.put("/products/{id}")
def replace_product(id: int, product: Product):
    if id not in db:
        raise HTTPException(status_code=404, detail="Product not found")
    db[id] = product
    return db[id]

@app.patch("/products/{id}")
def update_product(id: int, update: ProductUpdate):
    if id not in db:
        raise HTTPException(status_code=404, detail="Product not found")
    stored = db[id].dict()
    stored.update(update.dict(exclude_unset=True))
    db[id] = Product(**stored)
    return db[id]

# Q6
@app.delete("/products/{id}", status_code=204)
def delete_product(id: int):
    if id not in db:
```

```
        raise HTTPException(status_code=404, detail="Product not found")
    del db[id]
```

Section 2 — Q1 (terminal commands)

```
mkdir myapp && cd myapp
python -m venv venv
source venv/bin/activate          # (Windows: venv\Scripts\activate)
pip install fastapi "uvicorn[standard]"
# create main.py with a FastAPI() app + one route
uvicorn main:app --reload
```

Section 2 — Q4 (dependency injection auth)

```
from fastapi import FastAPI, Header, HTTPException, Depends

app = FastAPI()

def verify_token(authorization: str = Header(...)):
    if authorization != "Bearer secrettoken":
        raise HTTPException(status_code=401, detail="Invalid or missing token")
    return True

@app.get("/admin/data", dependencies=[Depends(verify_token)])
def get_admin_data():
    return {"secret": "only visible with a valid token"}
```